

在分布式环境下，如何确保多人并发新增班次时数据插入不发生冲突？根据业务需求，如何选择合适的分布式锁实现方案？

✎ 风格切换

💬 继续提问

📄

🔄

|

👍

👎

...

✕

思路分析：考虑到是新增的操作，并且新增失败了，我就先分析了接口日志，定位到失败请求的代码块和具体异常，猜测可能是同时插入一条数据，并且主键唯一约束所以导致了失败，就有数据库表结构，确认唯一约束与索引设计

事先知道是添加的操作，那么考虑到是不是由于多人同时操作导致数据覆盖的问题

时间

开班时间

选择时间

~ 选择时间

📅

搜索

培训老师姓名

请输入培训老师姓名

🔍

城市

全部

👇

状态

全部

👇

新增培训班次

📄 导出数据

问题就是多人并发的新增培训班次，由于班次主键id不可重复，有着唯一的约束，所以会导致插入数据失败；

解决方案就是加锁！！分布式锁，部署在多台机器的 单锁肯定是不可以的还是有冲突问题的呀！！

分布式锁

1. 什么是锁与为什么要用锁

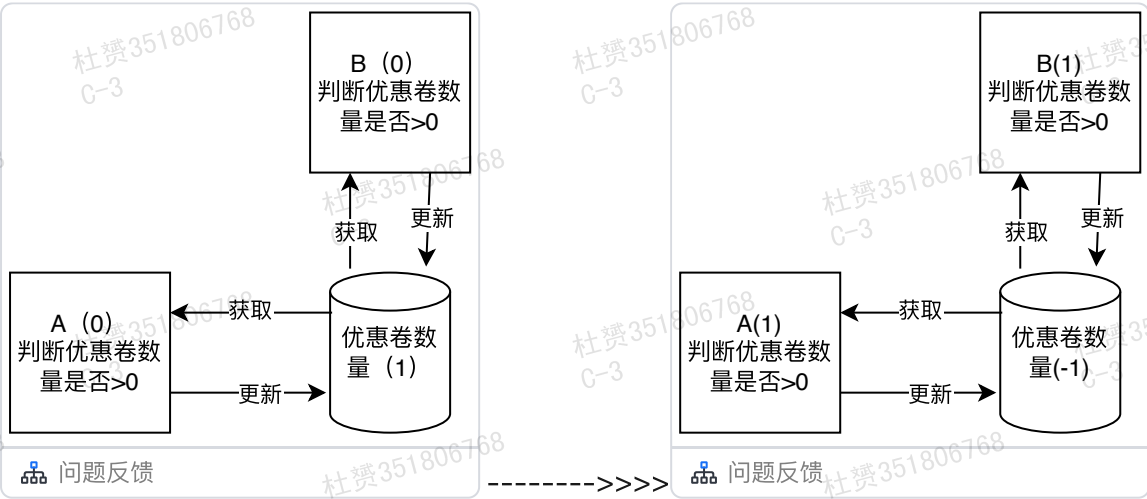
1.1 锁的定义

java当中的锁、是在多线程环境下为保证共享资源健康、线程安全的一种手段。

线程操作某个共享资源之前，先对资源加一层锁，保证操作期间没有其他线程访问资源，当操作完成后，再释放锁。

1.2 软件系统为什么要使用锁

以优惠券领取场景为例，A、B两个服务同时去数据库请求优惠券数量，如果优惠券数量>0，那么就允许优惠券发放，将优惠券数量减一并同步给数据库。



资源争抢时，锁是处理资源争抢的一种手段

- 在单进程的系统中，当存在多个线程可以同时改变某个变量（可变共享变量）时，就需要对变量或代码块做同步(synchronize)处理，使其在修改这种变量时能够线性执行消除并发影响。
- 同步操作可以通过锁来实现的。为了实现多个线程在一个时刻同一个代码块只能有一个线程可执行，那么需要在某个地方做个标记，这个标记必须每个线程都能看到，当标记不存在时可以设置该标记，其余后续线程发现已经有标记了则等待拥有标记的线程结束同步代码块取消标记后再去尝试设置标记。这个标记可以理解为止。
- 不同地方实现锁的方式也不一样，只要能满足所有线程都能看得到标记即可，例如以下三种不同的锁的形式：
 - 如 Java 中 synchronize 是在对象头设置标记。

-
- linux 内核中也是利用互斥量或信号量等内存数据做标记。
 - 或者使用某个文件是否存在作为锁等。只需要满足在对标记进行修改能保证原子性和内存可见性即可。

^ 点击展开内容

2. linux操作系统的三种锁

2.1 mutex(互斥锁)

一把确定的锁，解铃还须系铃人

互斥锁主要用于实现内核中的互斥访问功能。内核互斥锁是在原子API之上实现的，但这对于内核用户是不可见的。对它的访问必须遵循一些规则：同一时间只能有一个任务持有互斥锁，而且只有这个任务可以对互斥锁进行解锁。互斥锁不能进行递归锁定或解锁。一个互斥锁对象必须通过其API初始化，而不能使用memset或复制初始化。一个任务在持有互斥锁的时候是不能结束的。互斥锁所使用的内存区域是不能被释放的。使用中的互斥锁是不能被重新初始化的。并且互斥锁不能用于中断上下文。但是互斥锁比当前的内核信号量选项更快，并且更加紧凑，因此如果它们满足您的需求，那么它们将是您明智的选择

2.2 Spinlock(自旋锁)

持续保持轮训锁，避免反复切换，但是临界资源的代码执行要很短，自旋锁的效率远高于互斥锁

自旋锁的主要特征是使用者在想要获得临界区执行权限时，如果临界区已经被加锁，那么自旋锁并不会阻塞睡眠，等待系统来主动唤醒，而是原地忙轮询资源是否被释放加锁，自旋就是自我旋转，这个名字还是很形象的。自旋锁有它的优点就是避免了系统的唤醒，自己来执行轮询，如果在临界区的资源代码非常短且是原子的，那么使用起来是非常方便的，避免了各种上下文切换，开销非常小，因此在内核的一些数据结构中自旋锁被广泛的使用。

2.3 semaphore(信号量)

单机内核的版本控制（compare and set CAS)

linux内核的信号量在概念和原理上与用户态的SystemV的IPC机制信号量是一样的，但是它绝不可能在内核之外使用，因此它与SystemV的IPC机制信号量毫不相干。

信号量在创建时需要设置一个初始值，表示同时可以有几个任务可以访问该信号量保护的共享资源，初始值为1就变成互斥锁（Mutex），即同时只能有一个任务可以访问信号量保护的共享资源。一个任务要想访问共享资源，首先必须得到信号量，获取信号量的操作将把信号量的值减1，若当前信号量的值为负数，表明无法获得信号量，该任务必须挂起在该信号量的等待队列等待该信号量可用；若当前信号量的值为非负数，表示可以获得信号量，因而可以立刻访问被该信号量保护的共享资源。当任务访问完被信号量保护的共享资源后，必须释放信号量，释放信号量通过把信号量的值加1实现，如果信号量的值为非正数，表明有任务等待当前信号量，因此它也唤醒所有等待该信号量的任务。

2. 分布式锁

分布式锁顾名思义，就是控制分布式系统不同进程共同访问共享资源的一种锁的实现，如果有两个服务同时访问同一个资源，我们可以通过分布式锁来对这两个服务进行互斥限制。

2.1 分布式锁的存在方式

- 当在分布式模型下，数据只有一份（或理解为代码有访问限制），此时需要利用锁的技术控制某一时刻修改数据的进程数。
- 与单机模式下的锁不仅需要保证进程可见，还需要考虑进程与锁之间的网络问题。（分布式情况下之所以问题变得复杂，主要就是需要考虑到网络的延时和不可靠）
- 分布式锁还是可以将标记存在内存，只是该内存不是某个进程分配的内存而是在公共内存，如：
 - Redis
 - Memcache
 - 数据库
 - zookeeper文件夹

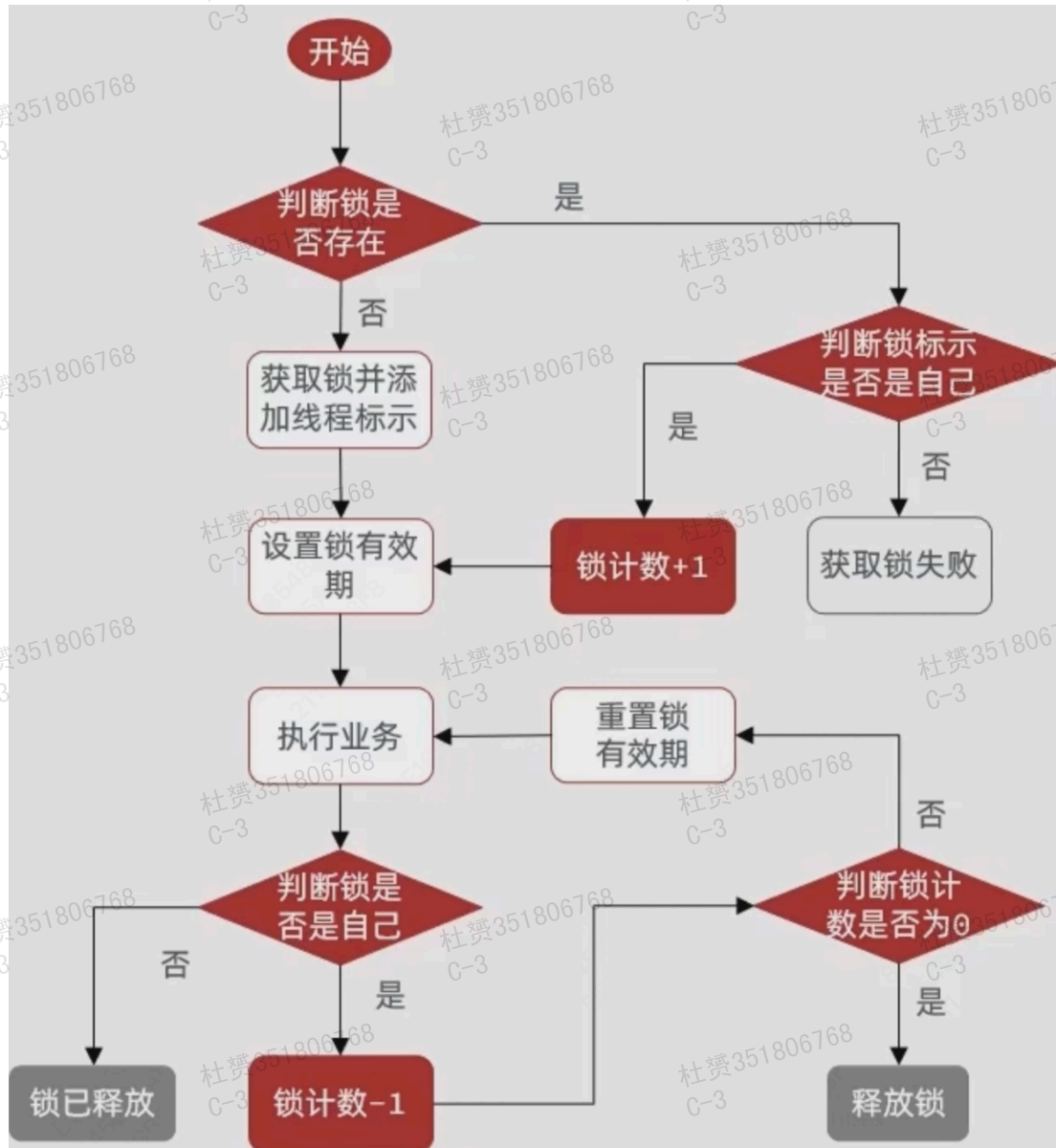
锁与单机的实现是一样的，只要保证标记能互斥就行。

点击展开内容

分布式锁要满足的业务要求

- 可以保证在分布式部署的应用集群中，同一个方法在同一时间只能被一台机器上的一个线程执行。
- 这把锁要是一把可重入锁（避免死锁）

点击展开内容



- 这把锁最好是一把阻塞锁（根据业务需求，考虑是否使用阻塞 or CAS)
- 这把锁最好是一把公平锁（根据业务需求考虑要不要这条，先到先得，后到等前面的)
- 有高可用的获取锁和释放锁功能
- 获取锁和释放锁的性能要好

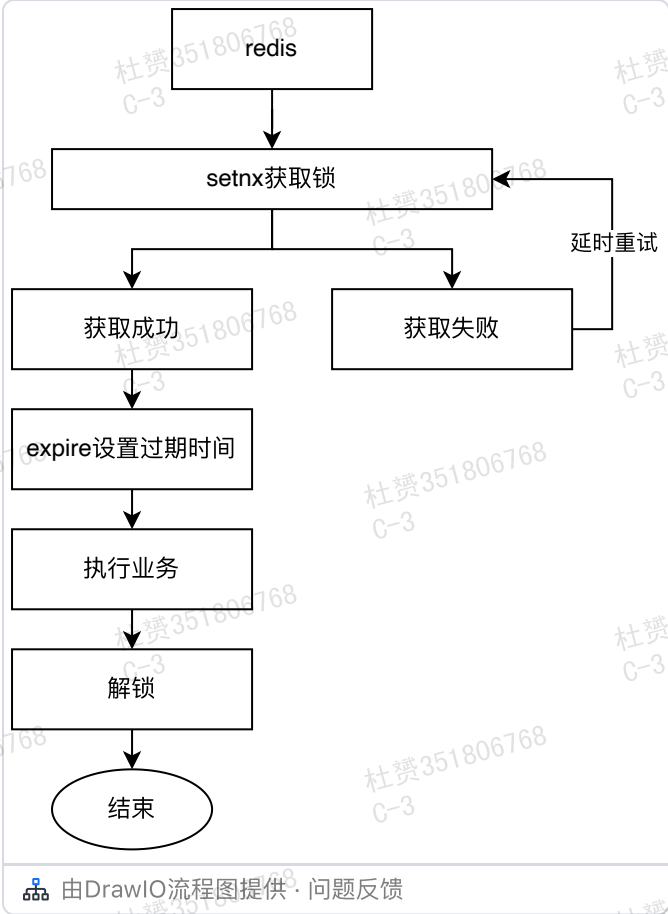
2.2 Redis实现

2.2.1 setnx和expire

setnx 的含义就是 SET if Not Exists，其主要有两个参数 setnx(key, value)。该方法是原子的，如果 key 不存在，则设置当前 key 成功，返回 1；如果当前 key 已经存在，则设置当前 key 失败，返回 0。

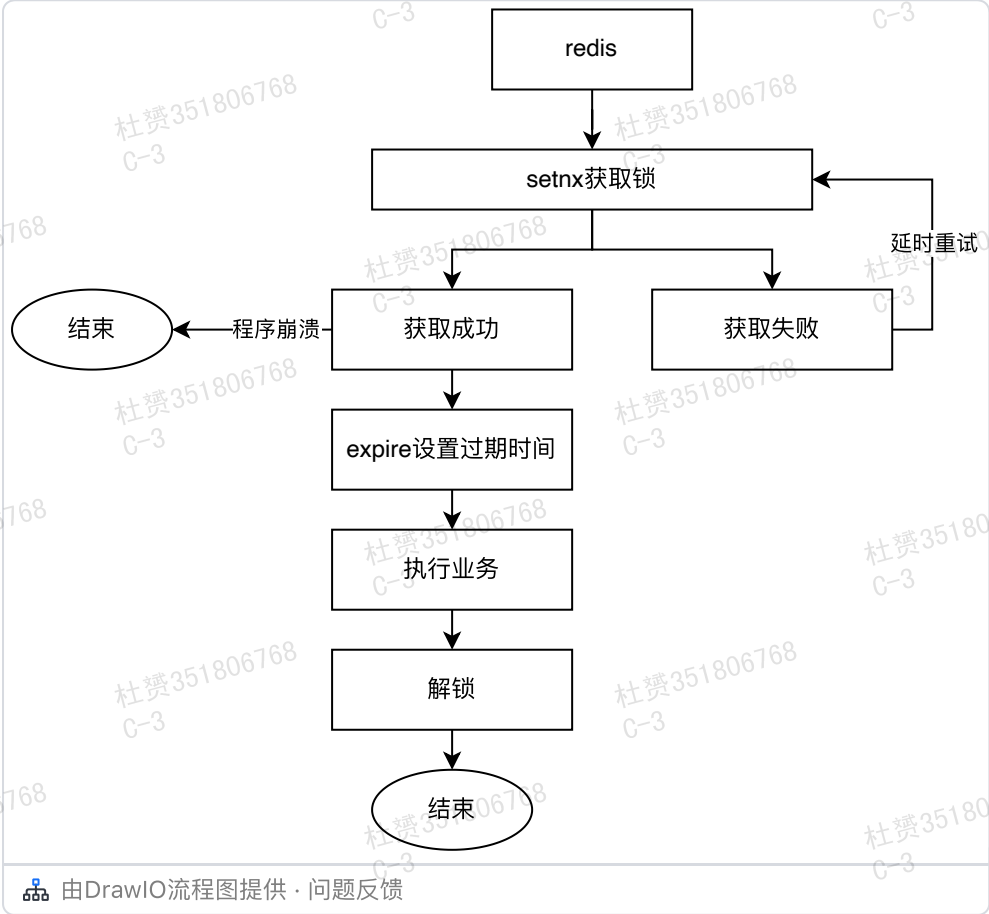
expire 设置过期时间，要注意的是 setnx 命令不能设置 key 的超时时间，只能通过 expire() 来对 key 设置。**expire() 命令对 lockkey 设置超时时间，为的是避免死锁问题。**

设计方案：



存在问题：

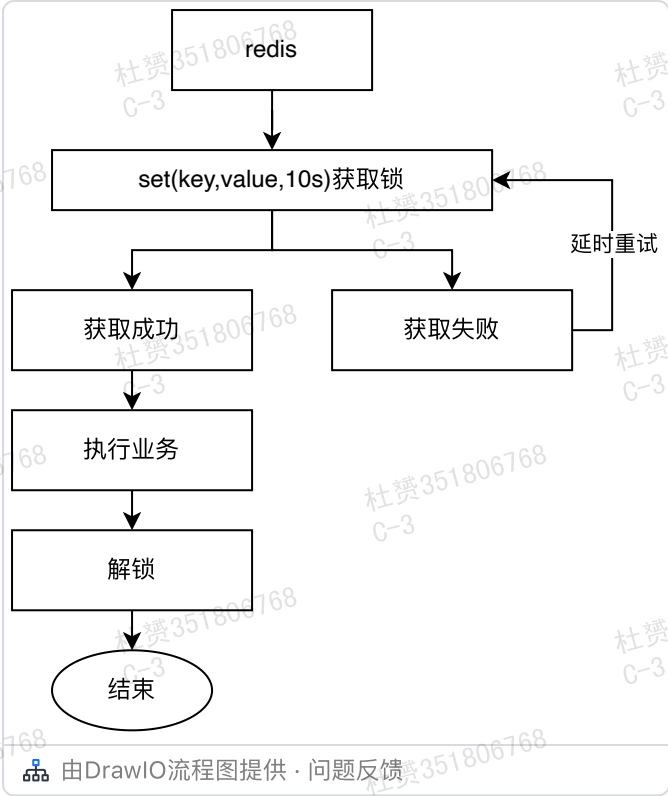
使用了两个命令，不是原子操作，在 setnx 执行完之后，如果系统突然重启或者崩溃，锁就没有设置过期时间，这样就会导致锁一直存在，其他线程就永远获取不到锁了。



2.2.2 使用Lua脚本/set ex px nx

设计方案:

Redis客户端保证lua脚本执行的原子性，另外我们也可以使用Redis的set的拓展指令 **SET key value[EX seconds][PX milliseconds][NX|XX]**来保证加锁的原子性。



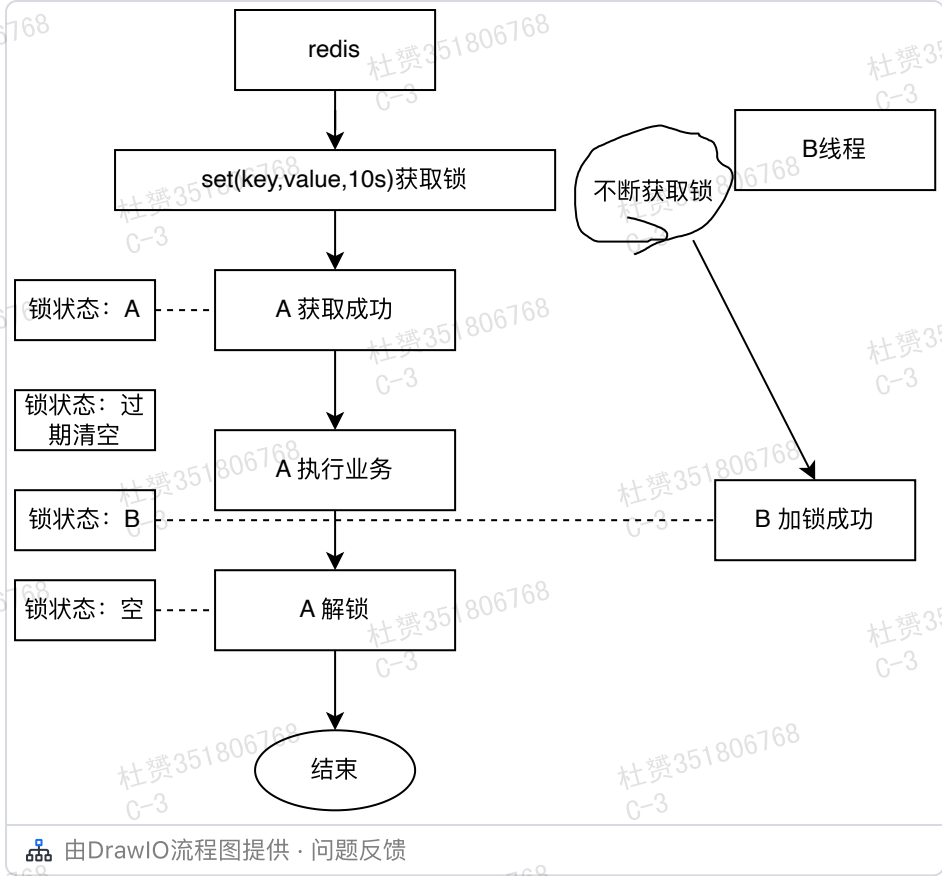
代码块

```
1 if redis.call('setnx',KEYS[1],ARGV[1]) == 1 then
2     redis.call('expire',KEYS[1],ARGV[2])
3 else
4     return 0
5 end;
```

Java

存在问题:

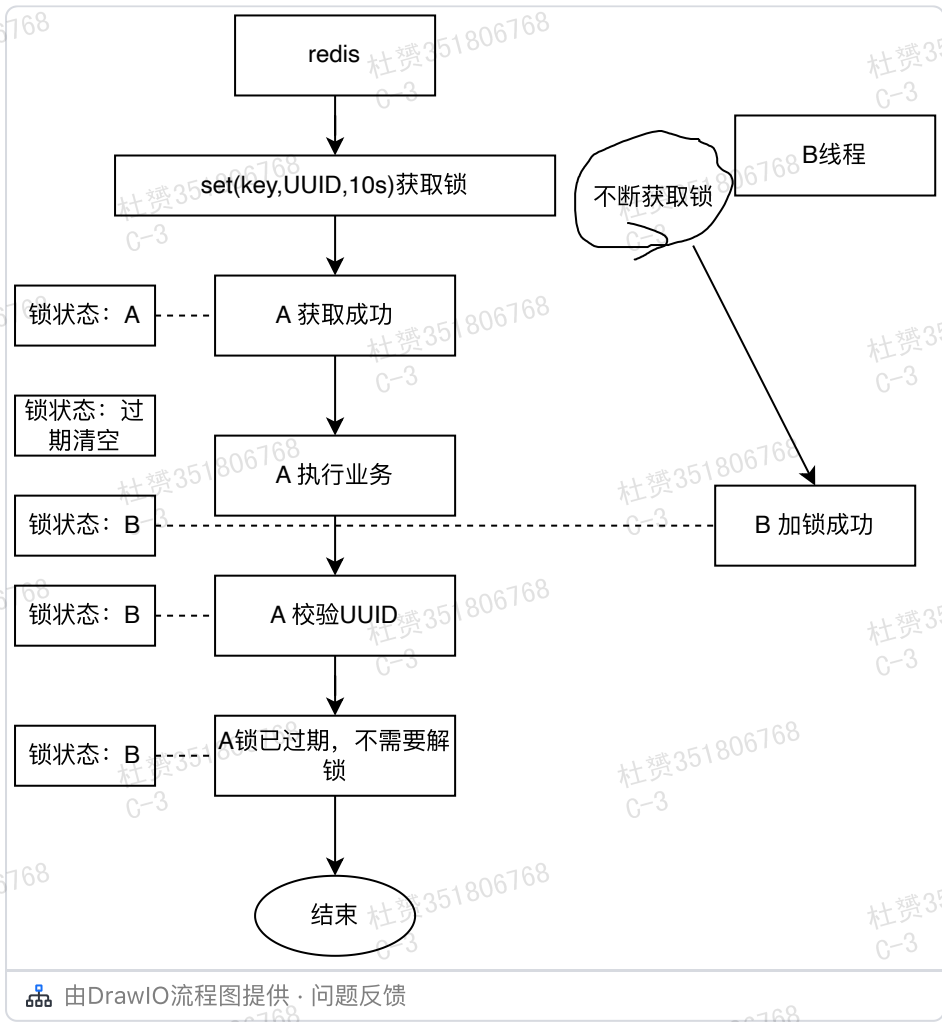
当A线程的锁过期之后，线程B获取锁成功，这时A线程业务执行完毕，到达解锁阶段，从redis中获取到了对应的锁，但是此时A获取的锁是B线程创建的，这就导致A把别人创建的锁删除了。



2.2.3 使用UUID

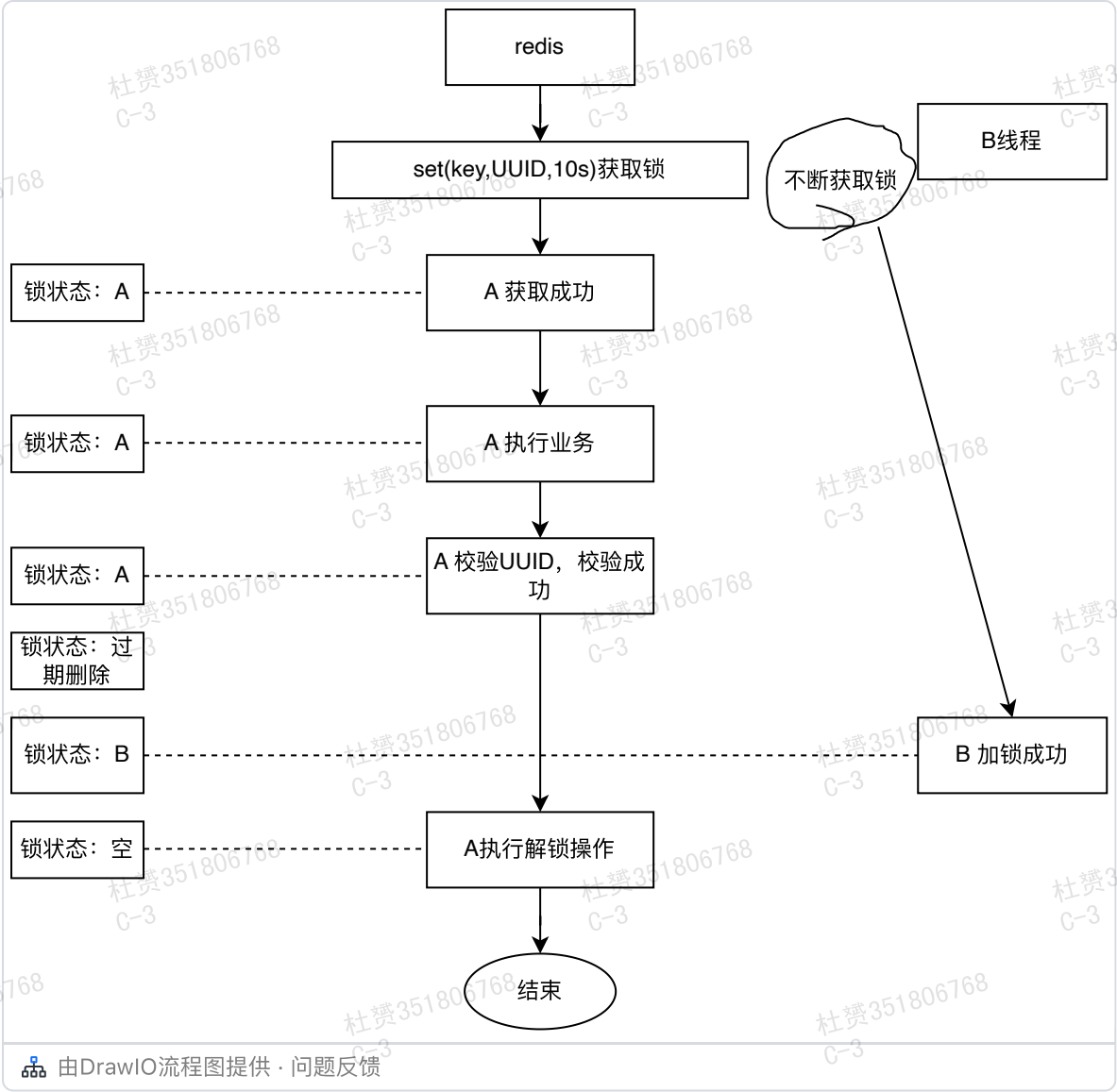
设计方案:

将value值设置为UUID，每次进行解锁操作的时候，都会比较UUID，从而保证只能删除本线程所创建的锁。



存在问题:

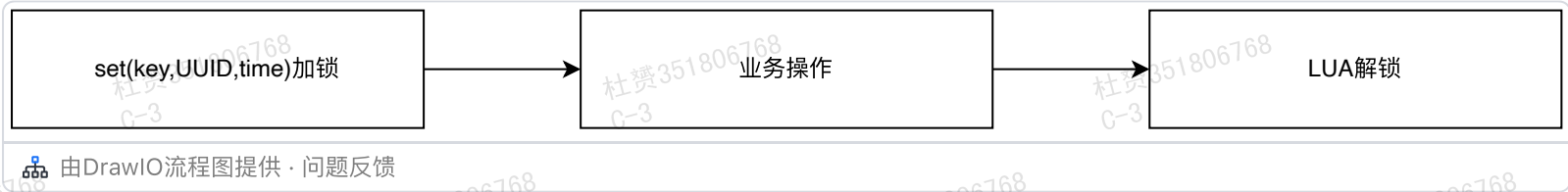
解锁行为其实分为两步，第一步首先get锁，第二步是校验UUID是否相同。这样就会出现一种情况，当Aget锁的时候，获取到的UUID是本线程所设置的，然后下一时刻锁过期删除了，这时另一个线程B发现锁没有被占用了，就会创建自己的锁，然后A判断完UUID之后，执行删除操作，这是A线程同样把B线程创建的锁删除了。



2.2.4 解锁操作原子化

设计方案：

使用Lua脚本来保证解锁操作原子化。



代码块 Lua

```
// 判断锁是自己的，才释放
if redis.call("GET",KEYS[1]) == ARGV[1]
then
return redis.call("DEL",KEYS[1])
else
return 0
```

存在问题：

在加锁的时候，我们不知道业务需要执行多少时间，加锁的时间不好评估，如果加锁时间设置的过少，就会导致业务上的风险。有个妥协的方案：将过期时间尽可能的设置长一点，这样虽然能解决问题，但是也不够完美。

^ 点击展开内容

锁的优化的操作可以看：

基于 Redlock 做分布式锁

Redlock 是 Redis 的作者 antirez 给出的集群模式的 Redis 分布式锁，它基于 N 个完全独立的 Redis 节点（通常情况下 N 可以设置成 5）。

基于 redisson 做分布式锁

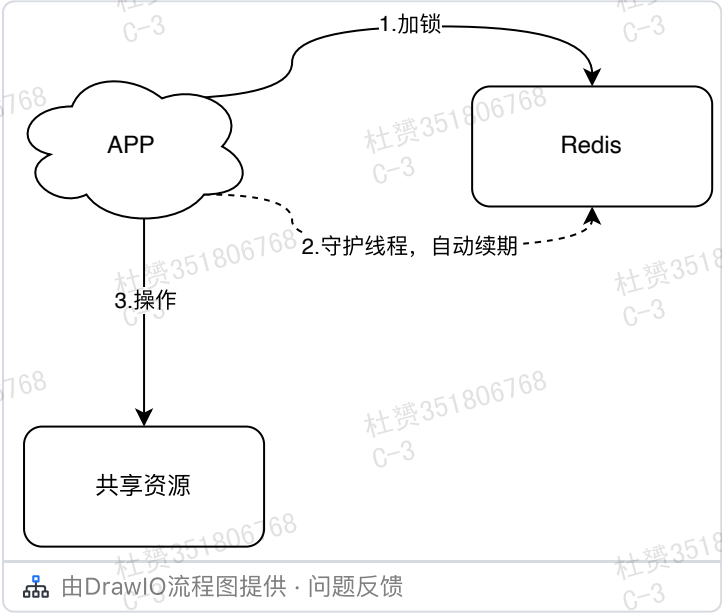
redisson 是 redis 官方的分布式锁组件。GitHub 地址：<https://github.com/redisson/redisson>

2.2.5 Redisson如何实现加锁时间的控制

设计方案：

Redisson是java语言实现的 Redis SDK 客户端，当我们使用分布式锁的时候，它使用**自动续期**来解决加锁时间的问题。

自动续期是指在加锁的时候，开启一个守护线程来监控锁的过期时间，当锁快过期的时候，操作共享资源还没完成，那么就会重设锁的过期时间，从而达到延长锁过期时间的效果。



2.3 ZooKeeper 实现

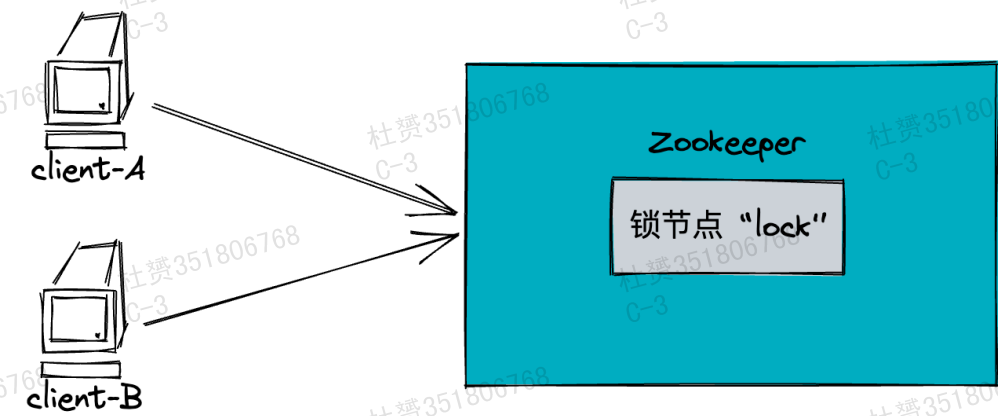
zookeeper提供了监听存储在zk内部数据的功能，从而可以达到基于数据的集群管理；以创建临时带序号的节点，且ZK会保证节点的全局有序性；利用上面两个特性，可以利用ZK实现分布式锁的功能。

2.3.1 zookeeper 锁相关基础知识

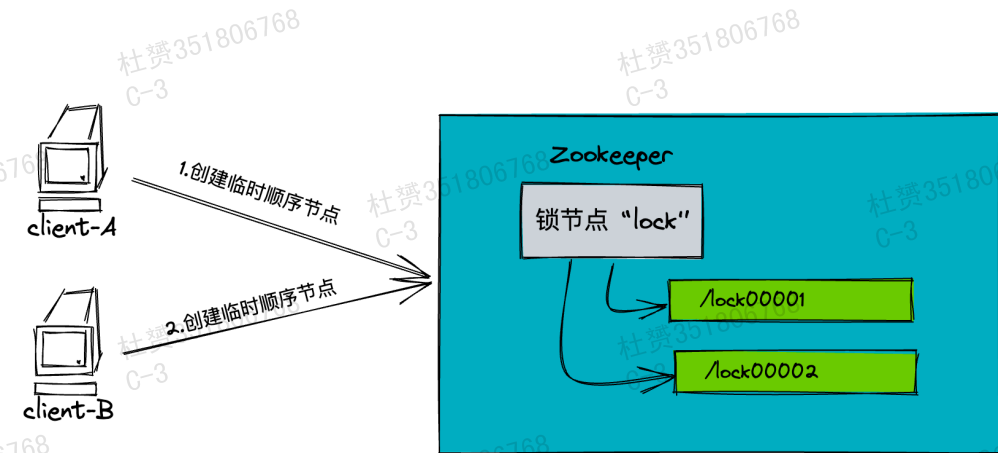
- zk 一般由多个节点构成（单数），采用 zab 一致性协议。因此可以将 zk 看成是一个单点结构，对其修改数据其内部自动将所有节点数据进行修改而后才提供查询服务。
- zk 的数据以目录树的形式，每个目录称为 znode，znode 中可存储数据（一般不超过 1M），还可以在其中增加子节点。
- 子节点有三种类型。序列化节点，每在该节点下增加一个节点自动给该节点的名称上自增。临时节点，一旦创建这个 znode 的客户端与服务器失去联系，这个 znode 也将自动删除。最后就是普通节点。
- Watch 机制，client 可以监控每个节点的变化，当产生变化会给 client 产生一个事件。

2.3.2 如何实现分布式锁

- 利用Zookeeper可以创建临时带序号节点的特性来实现一个分布式锁
- 多个系统的多个线程都要在此目录下创建临时的顺序节点，因为Zk会为我们保证节点的顺序性，所以可以利用节点的顺序性来对不同线程的先来后到进行判断。此时，在分布式下应该添加锁的线程就是zk指定目录下序号最小的临时序列节点所代表的线程。
- 当每个线程请求对应资源时，都需要先在zk指定目录下创建临时顺序节点，然后获取当前目录下最小的节点序号(加锁的线程)，判断最小节点是不是当前节点(判断是不是自己)，如果是那么获取锁成功，如果不是那么获取锁失败。
- 获取锁失败的线程获取当前节点上一个临时顺序节点，并对对此节点进行监听。当该节点删除的时候(上一个线程执行结束删除或者是掉线zk删除临时节点)这个线程会获取到通知，代表获取到了锁。**注意这里每个节点都是只监听它的上一个节点即可，而不是监听序号最小的节点。因为临时节点都是带序号的，而且序号不会回退，所以只需要 监听比其小1的节点。只要比它小1的节点被清除，则它就可以 拿到资源去操作**



在上图中， ZooKeeper里面有一把锁节点 Lock，这个锁就是 ZooKeeper的一个节点，当两个客户端来获取这把锁的时候，会对 ZooKeeper进行加锁的请求，也就是我们所说的 临时顺序节点。



当我们在 /Lock目录下创建了一个顺序临时节点后，ZK会自动对这个临时节点维护 一个节点序号，并且这个节点是递增的，比如我们 clientA 创建了一个临时顺序节点，ZK内部会生成一个序号：/lock0000000001，那么 clientB 也生成了一个临时顺序节点，ZK会生成一个序号为 /lock0000000002，在这里数字都是依次递增的，从1开始递增，ZK内部会维护这个顺序。



这时候，ClientA和ClientB会进行监听判断，在父节点下，我是不是最小的，如果是的话，那么就可以加锁了。



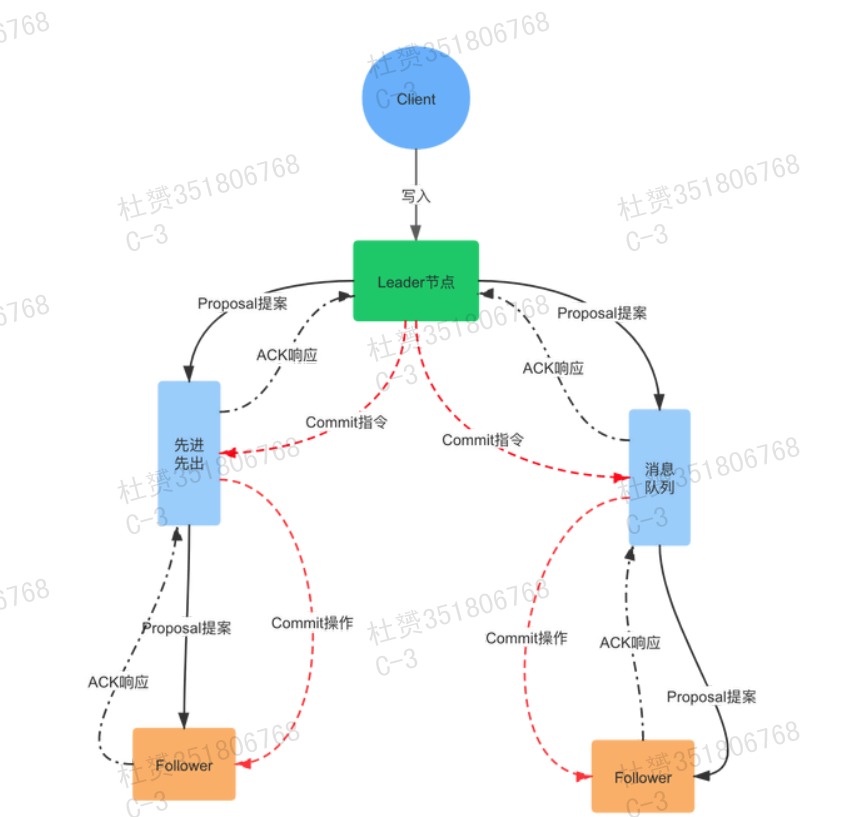
加锁失败呀，clientB 就会按照顺序去监听前一个节点(clientA)，是否完成工作了，如果完成了，clientB才可以进行加锁工作



当clientA被删除以后，就表示有人释放了锁，这个时候就会通知 clientB重新去获取锁。这个时候clientB重新获取锁的时候，发现自己就是当前父节点下面最小的那个，于是clientB就开始加锁、工作等一系列操作。

2.3.3 为什么zookeeper 锁更安全

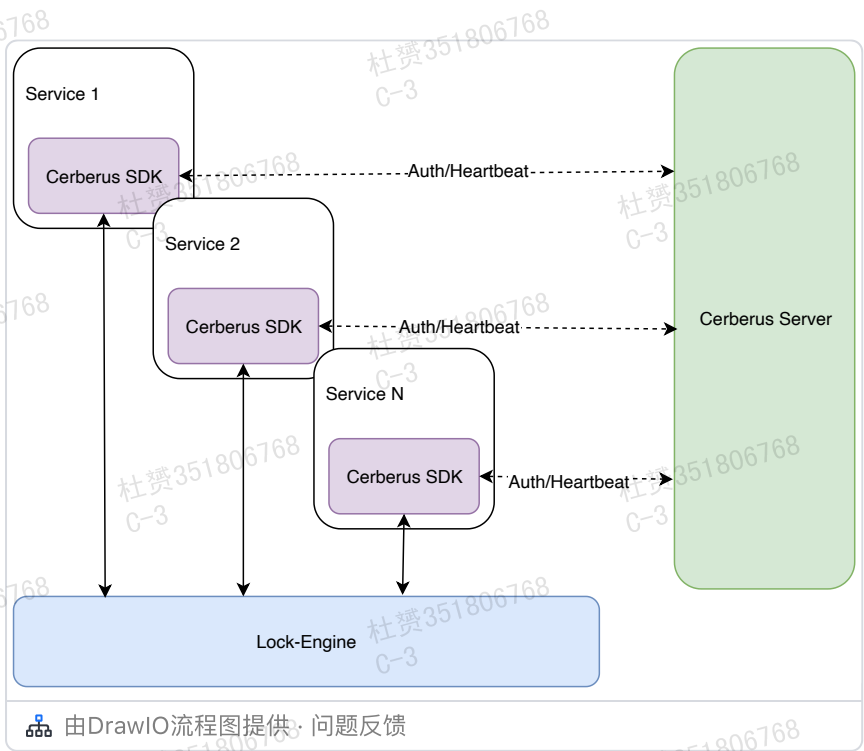
1. 使用Squirrel/Cellar集群；Cellar和Squirrel只保证最终一致性，副本间的数据复制是异步进行，主从切换之后可能有部分数据没有复制过去可能会丢失锁情况，故一致性要求高的业务不推荐使用Cellar和Squirrel引擎，推荐使用zk。
2. 使用ZooKeeper集群，ZooKeeper通过ZAB协议实现数据一致性，确保分布式场景中的数据更新在集群中是有序、一致和可靠的。Cerberus底层依赖于ZooKeeper的临时节点，临时节点的生命周期在Client与集群的Session结束时结束。因此如果某个Client节点存在网络问题，与ZooKeeper集群断开连接，Session超时同样会导致锁被错误的释放（导致被其他线程错误地持有），因此ZooKeeper也无法完全保证线性一致。



1. 上述2点的区别在于ZooKeeper存在不一致的概率更低，对于一致性要求不高的程序可以行取舍。要求线性一致的，务必自行提供降级方案，即加锁错误时业务逻辑仍要自行保证一致性。

3. 公司的分布式锁组件

Cerberus - 分布式锁



3.1 Cerberus Server(管理端)

Cerberus 管理端提供以下功能：

1. 新用户接入申请/审批流程
2. 管理端远程更新SDK锁引擎配置信息
3. 展示已接入的服务列表及详情
4. 展示应用下各实例的锁列表与运行状态 ^[2]，以及运维操作
5. 提供SDK用于鉴权的功能
6. 提供接收SDK心跳的接口，返回值包含用于控制SDK状态的配置信息 ^[3]

3.2 Lock-Engine

可配置

- redis
- cellar
- zookeeper

Lock-engine提供分布式锁的核心逻辑，主要控制：

- 1.锁的排他性，保证一个锁在某时间段只会被一个线程持有
- 2.锁的超时机制，保证持有锁的线程出现意外（例如程序意外停止，宕机）时，锁不会被此线程永久占用

3.3 Cerberus SDK

Cerberus SDK提供以下功能：

1. 业务接口：Cerberus SDK向业务提供操作接口：DLM Api ^[4]，Service通过Cerberus SDK提供的接口使用分布式锁功能
2. 引擎交互：调用Lock-Engine实现分布式锁各接口的底层逻辑
3. SDK鉴权：Cerberus SDK在启动时调用管理端鉴权接口校验使用权限 ^[5]
4. 心跳探活：Cerberus SDK定时（5s）向管理端发送心跳（以调用服务端接口形式） ^[2]
5. 远程配置：Cerberus SDK通过心跳接口响应Cerberus Server的远程控制，依据接口返回的配置信息改变SDK相应的状态 ^[3]

3.4 FAQ

1. Cerberus相比开源方案有什么优势？

1.1 方案调研

分布式锁是一个在分布式环境中的重要原语，它表明不同进程间采用互斥的方式操作共享资源。常见的场景是作为一个sdk被引入到大型项目中，主要解决两类问题：

- 提升效率：加锁是为了避免不必要的重复处理。例如防止幂等任务被多个执行者抢占。此时对锁的正确性要求不高；
- 保证正确性：加锁是为了避免Race Condition导致逻辑错误。例如直接使用分布式锁实现防重，幂等机制。此时如果锁出现错误会引起严重后果，因此对锁的正确性要求高。

下面分社区与工业界两个维度分别介绍。

1.1.1 社区调研

1.1.1.1 Menagerie

Menagerie是一个使用ZooKeeper实现的Java Concurrency Libraries。除了实现分布式锁，Menagerie实现了Java并发包下的大多数功能， 例如java.util.concurrent.locks.Condition，java.util.concurrent.BlockingQueue等。Menagerie存在的问题是并发效率不高， 例如每次获取锁时都必须先获取ZooKeeper Session， 在Menagerie中这是一个synchronized方法。

1.1.1.2 Apache Helix

Apache Helix是一个基于一致性协议的集群资源管理框架，其中提供了分布式锁功能。与传统的ZooKeeper实现方式相比， Apache Helix更进一步实现了公平的锁获取机制：

- 避免锁饥饿问题，各个节点获取锁的机会均等。
- 在节点失效时该节点持有的锁会被剩余节点机会均等地取得。
- 有新节点加入时会触发再均衡机制，保证各节点机会均等。

Helix内部同样使用ZooKeeper的临时节点机制实现， 但Helix的特性在于保证锁的“公平性”， 因此实现方式比Curator更复杂，以牺牲性能为代价。

1.1.1.3 Redisson

Redisson是架设在Redis基础上的一个java驻内存数据网格（In-Memory Data Grid）。使得原本作为协调单机多线程并发程序的工具包获得了协调分布式多机多线程并发系统的能力，降低了设计和研发大规模分布式系统的难度。同时结合各富特色的分布式服务，更进一步简化了分布式环境中程序相互之间的协作。提供联锁，读写锁，公平锁，红锁，信号量，可过期性信号量和闭锁这些实际当中对多线程高并发应用至关重要对基本部件。

1.1.2 业界调研

公司	场景	技术方案
Google:Chubby	Chubby是一套分布式协调系统，内部使用Paxos协调Master与Replicas。 Chubby lock service被应用在GFS, BigTable等项目中，其首要设计目标是高可靠性，而不是高性能。 Chubby被作为粗粒度锁使用，例如被用于选主。持有锁的时间跨度一般为小时或天，而不是秒级。	Chubby对外提供类似于文件系统的API，在Chubby创建文件路径即加锁操作。Chubby使用Delay和SequenceNumber来优化锁机制。Delay保证客户端异常释放锁时，Chubby仍认为该客户端一直持有锁。Sequence number 指锁的持有者向Chubby服务端请求一个序号（包括几个属性），然后之后在需要使用锁的时候将该序号一并发给 Chubby 服务器，服务端检查序号的合法性，包括 number 是否有效等。
京东SharkLock	SharkLock是基于Redis实现的分布式锁。	锁的排他性由SETNX原语实现，使用timeout与续租机制实现锁的强制释放。
蚂蚁金服SOFAJRaft-RheaKV 分布式锁	RheaKV 是基于 SOFAJRaft 和 RocksDB 实现的嵌入式、分布式、高可用、强一致的 KV 存储类库。	RheaKV对外提供lock接口，为了优化数据的读写，按不同的存储类型，提供不同的锁特性。RheaKV提供wathcdog调度器来控制锁的自动续租机制，避免锁在任务完成前提前释放，和锁永不释放造成死锁。
Netflix: Curator	Curator是ZooKeeper的客户端封装，其分布式锁的实现完全由ZooKeeper完成。	在ZooKeeper创建EPHEMERAL_SEQUENTIAL节点视为加锁，节点的EPHEMERAL特性保证了锁持有者与ZooKeeper断开时强制释放锁；节点的SEQUENTIAL特性避免了加锁较多时的惊群效应。

横向对比上述方案，分布式锁主要解决两个问题：

- a. 保证锁的排他性，即同一个锁在被持有的时间段内只能被一个线程持有；
- b. 锁超时机制，保证持有锁的线程出现异常时（例如Client失效，宕机等），锁不会被永久占用。

因此可以得到以下矩阵：

No	类别	排他机制	锁超时机制	缺陷	举例
1	基于分布式一致性算法	基础方案： 获取锁时新建临时节点（或租约节点），视为该节点对应的锁已被占用。	Client与集群断开时，临时节点自动消失。视为锁被释放。	羊群效应。	使用ZooKeeper，etcd，或者实现Raft算法。
		ZooKeeper避免羊群效应的优化方案： a. 新建“/locknode/lock-“的EPHEMERAL_SEQUENTIAL节点NodeA； b. 在路径“/lockNode/”调用getChildren()，注意不要加锁； c. 如果NodeA在childrenList里是最小节点（拥有最小数字后缀）说明获取锁成功，结束并返回。 d. 如果NodeA不是最小节点，使用exists()在后缀数值仅次于NodeA的节点加Watcher。 e. 如果exists()返回false，进入第二步重试；如果返回true则需要等待Watcher触发，再进入第二步重新尝试获取锁。			
2	基于数据库	排他锁方案： 获取锁时新增一条记录，并对锁对应的行加排他锁，阻塞其他试图获取该锁的请求。	Client失效时，数据库会有超时机制保证释放锁。	如果多个线程同时间长时间持有锁，会导致数据库连接过多。	使用Mysql等。
		唯一键方案： 建立具有唯一字段的表。获取锁时向表中新增一条数据，如果成功表示加锁成功，如果出现唯一键冲突说明对应的锁已被占用。	设置超时阈值，由定时任务定期删除表内记录。	如果持有锁的时间区间太长会被误删，导致锁错误地进入释放状态。	
3	基于K-V存储	直接使用Redis集群： 使用SETNX原语加锁，如果失败表明该锁已被其他线程持有。	由expireTime保证超时机制。如果Client超过expireTime仍未主动释放锁，Redis会使对应的key过期，强制释放锁。	a. 如果持有锁的时间区间太长会被误删，导致锁错误地进入释放状态。	Redis，Squirrel等。
		续租方案： 如果超时时间设置太长，持有锁的Client宕机时其他Client无法尽快获取到锁。因此针对第一种方案的优化，采取续租机制。 即Client在获取锁时设置较短的超时时间，然后在持有锁期间定时刷新超时时间。		b. Redis集群设计初衷在于性能而不在于数据一致性。如果Redis丢失数据将导致导致锁错误地进入释放状态。	

	间，直到释放锁为止。		
	Redlock方案： 用于解决Redis导致的数据不一致问题。直接连接到多个Redis master(例如5个)，加锁时同时向所有master请求，得到其中大部分（最少3个）回应success时视为加锁成功。 redlock实现锁比直接使用Redis集群更可靠。	与Redis相同	a. 如果持有锁的时间区间太长会被误删，导致锁错误地进入释放状态。 b. 性能低于直接使用Redis。

1.1.2 调研结论

针对分布式锁的使用场景以及主流实现方案可知， 分布式锁的QPS、并发性能与锁的逻辑正确性高度依赖底层的数据一致性机制；分布式领域线性一致一般以牺牲性能为代价。因此，为满足业务差异化的需求，Cerberus提供两种Lock-Engine分别基于ZooKeeper和Squirrel，业务方依据自身场景选择合适的解决方案。其中Squirrel主要用于满足高性能的场景；ZooKeeper用于满足强一致、高可用的场景。

Cerberus的优势体现在：

- 1. 多引擎：为兼顾了不同业务场景差异化需求，Cerberus提供了多种锁引擎。经调研可知，分布式锁的三种主流实现：基于数据库、基于ZooKeeper、基于Redis在高性能与强数据一致性上无法做到兼顾。而美团内部业务方（研发）面临的场景是多种多样的，因此Cerberus目前提供了两种Lock-Engine的实现，基于ZooKeeper与Squirrel。视业务需要，高一致性要求选用前者，高性能选用后者。
- 2. 易接入：相比与开源方案普遍只提供一套SDK，Cerberus还提供了一个服务端（Cerberus Server）用于管理SDK鉴权，方便用户接入。
- 3. 高可用：Cerberus Server可以通过Cerberus SDK的心跳机制控制各个SDK的启/禁用，在Lock-Engine使用的ZooKeeper与Squirrel集群失效时临时关闭锁服务并切换集群。经调研，目前的开源项目均无此功能。

2. Cerberus如何处理高一致性要求的场景？

目前Cerberus实现了基于ZooKeeper的Lock-Engine，用于在一致性要求高的场景下提供分布式锁服务；

3. Cerberus如何处理高性能要求的场景？

目前Cerberus实现了基于Squirrel的Lock-Engine，用于处理优先考虑高吞吐量需求的业务场景。

4. 如何理解分布式锁的一致性保证？

- a. 使用Squirrel/Cellar集群；Cellar和Squirrel只保证最终一致性，副本间的数据复制是异步进行，主从切换之后可能有部分数据没有复制过去可能会丢失锁情况，故一致性要求高的业务不推荐使用Cellar和Squirrel引擎，推荐使用zk。
- b. 使用ZooKeeper集群，Cerberus底层依赖于ZooKeeper的临时节点，临时节点的生命周期在Client与集群的Session结束时结束。因此如果某个Client节点存在网络问题，与ZooKeeper集群断开连接，Session超时同样会导致锁被错误的释放（导致被其他线程错误地持有），因此ZooKeeper也无法完全保证线性一致。
- c. 上述2点的区别在于ZooKeeper存在不一致的概率更低，对于一致性要求不高的程序可以行取舍。要求线性一致的，务必自行提供降级方案，即加锁错误时业务逻辑仍要自行保证一致性。

5. 有线性一致性要求的系统是否有必要分布式锁？

对于线性一致性要求的系统，分布式锁仍有适用场景。比如为了保证强一致性而采用了性能代价很高的操作，此时可以用分布式锁初筛一下，过滤掉绝大多数的重复请求，大幅提升系统处理性能。

3.5 业界对比

公司	场景	技术方案
Google:Chubby	Chubby是一套分布式协调系统，内部使用Paxos协调Master与Replicas。 Chubby lock service被应用在GFS，BigTable等项目中，其首要设计目标是高可靠性，而不是高性能。	Chubby对外提供类似于文件系统的API，在Chubby创建文件路径即加锁操作。Chubby使用Delay和SequenceNumber来优化锁机制。Delay保证客户端异常释放锁时，Chubby仍认为该客户端一直持有锁。 Sequence number 指锁的持有者向Chubby服务端请求一个序号（包括

	Chubby被作为粗粒度锁使用，例如被用于选主。持有锁的时间跨度一般为小时或天，而不是秒级。	几个属性)，然后之后在需要使用锁的时候将该序号一并发给 Chubby 服务器，服务端检查序号的合法性，包括 number 是否有效等。
京东SharkLock	SharkLock是基于Redis实现的分布式锁。	锁的排他性由SETNX原语实现，使用timeout与续租机制实现锁的强制释放。
蚂蚁金服SOFAJRaft-RheaKV 分布式锁	RheaKV 是基于 SOFAJRaft 和 RocksDB 实现的嵌入式、分布式、高可用、强一致的 KV 存储类库。	RheaKV对外提供lock接口，为了优化数据的读写，按不同的存储类型，提供不同的锁特性。RheaKV提供wathcdog调度器来控制锁的自动续租机制，避免锁在任务完成前提前释放，和锁永不释放造成死锁。
Netflix: Curator	Curator是ZooKeeper的客户端封装，其分布式锁的实现完全由ZooKeeper完成。	在ZooKeeper创建EPHEMERAL_SEQUENTIAL节点视为加锁，节点的EPHEMERAL特性保证了锁持有者与ZooKeeper断开时强制释放锁；节点的SEQUENTIAL特性避免了加锁较多时的惊群效应。

4. 分布式锁选择

优缺点及适用场景：

实现方式	优点	缺点	适用场景
Redis	1. 实现指令性能较高	1. 实现较为复杂 2. 需要额外多个服务来保障 3. 安全性较低 4. 随着并发和业务数量的提升其响应时间会有明显上升	1. 响应时间敏感 2. 高并发的分布式锁实现 3. qps (0, 1000]
Cellar	1. 指令性能较好 2. 在业务数量较高时有着不错的响应延迟	1. 实现较为复杂 2. 需要额外多个服务来保障 3. 安全性较低 4. 响应时间比Redis稍高	1. 高并发的分布式锁实现 2. qps (0, 1000]
Zookeeper	1. 现有的框架，实现简单 2. 等待锁队列，提升抢占锁效率	1. 添加和删除节点性能较低 2. 随着并发和业务数量提升响应时间和qps会明显下降	1. 并发量小，安全性要求较高的业务场景 todo：为啥会高 2. qps (0, 300]

简单选择矩阵：

	Cellar	Squirrel	Zk
响应时间敏感	✓	✓	
并发量高	✓	✓	

需要读写锁			✓
需要公平锁			✓
需要非公平锁	✓	✓	

场景

代码块

Java

```
1 private ApiResponse<Boolean> getAdd0perationApiResponse(RecruitmentTrainSessionModifyReq recruitmentTrainSessionModifyReq) {
2     if(recruitmentTrainSessionModifyReq.getCityId() == null){
3         return ApiResponse.fail(BusinessStatusEnum.PARAM_ERROR, "城市不能为空");
4     }
5
6     try {
7         String lockKey = SESSION_MODIFY_LOCK_KEY + "_" + recruitmentTrainSessionModifyReq.getCityId();
8         return redisLockUtils.tryLock(lockKey, () -> {
9             InsertRecruitmentSessionReq insertRecruitmentSessionReq = new InsertRecruitmentSessionReq();
10            buildInsertRecruitmentSessionReq(insertRecruitmentSessionReq, recruitmentTrainSessionModifyReq);
11            TechnicianResp<Boolean> booleanTechnicianResp = null;
12            try {
13                booleanTechnicianResp = doorToDoorManageLeadInputService.insertRecruitmentSession(insertRecruitmentSessionReq);
14                if (booleanTechnicianResp == null || booleanTechnicianResp.respFail()) {
15                    return ApiResponse.fail(BusinessStatusEnum.ERROR, "新增失败");
16                }
17            } catch (Exception e) {
18                log.error("doorToDoorManageLeadInputService.insertRecruitmentSession error", e);
19                if (booleanTechnicianResp != null) {
20                    return ApiResponse.fail(BusinessStatusEnum.ERROR, booleanTechnicianResp.getMsg());
21                }
22            }
23            return ApiResponse.success(true);
24        });
25    } catch (BizException e) {
26        log.error("RecruitmentTrainSessionModifyHandler acquire lock error", e);
27        return ApiResponse.fail(BusinessStatusEnum.ERROR, "请稍后重试");
28    }
29 }
30
31
32
33
34 /**
35  * 分布式锁工具类
36  * <p>
```

```
37 * caution: 使用该工具时, 给自己的 key 加上一个业务前缀 (方法的全限定名也可), 最大限度的避免大家撞 key
38 *
39 * @author xiangrui
40 */
41 @Component
42 @Slf4j
43 public class RedisLockUtils {
44
45     private static final String RETRY_LATER_MESSAGE = "请稍后重试";
46
47     private static final String DUPLICATE_LOCK_KEY_MESSAGE = "tryLock duplicate key:";
48
49     private static final String ERROR_DISTRIBUTED_LOCK_KEY_NOT_PROVIDED = "创建分布式锁时未传入key";
50
51
52
53     private static final String CAT_TYPE = RedisLockUtils.class.getSimpleName();
54
55     /**
56      * 默认过期时间为一分钟
57      */
58     public static final int DEFAULT_EXPIRY_TIME_MILLIS = 60;
59
60     @Autowired
61     private IDistributedLockManager distributedLockManager;
62
63     public <T> T tryLock(String key, Func2<T> func2) {
64         if (StringUtils.isEmpty(key)) {
65             throw new BizException(ERROR_DISTRIBUTED_LOCK_KEY_NOT_PROVIDED);
66         }
67         Lock lock = null;
68         try {
69             Integer timeout = Lion.getInt("com.sankuai.lifeevent.serviceprovider", "redis.lock.timeout", 30);
70             lock = distributedLockManager.getReentrantLock(key, DEFAULT_EXPIRY_TIME_MILLIS);
71             if (Objects.isNull(lock)) {
72                 log.error("tryLock getReentrantLock is null, key:{}", key);
73                 throw new BizException(RETRY_LATER_MESSAGE);
74             }
75             if (lock.tryLock(timeout, TimeUnit.SECONDS)) {
76                 return func2.invoke();
77             } else {
78                 Cat.logEvent(CAT_TYPE, DUPLICATE_LOCK_KEY_MESSAGE + key);
79                 log.warn(DUPLICATE_LOCK_KEY_MESSAGE + key);
80                 throw new BizException(RETRY_LATER_MESSAGE);
81             }
82         } catch (InterruptedException e) {
```

```
83     log.error("tryLock error, key:{}", key, e);
84     Thread.currentThread().interrupt();
85     throw new BizException(RETRY_LATER_MESSAGE);
86 } catch (Exception e) {
87     log.error("tryLock error, key:{}", key, e);
88     throw new BizException(RETRY_LATER_MESSAGE);
89 } finally {
90     if (Objects.nonNull(lock)) {
91         lock.unlock();
92     }
93 }
94 }
95
96 /**
97  * @param timeout 获取锁等待超时时间, 单位: 秒
98  */
99 public void tryLock(String key, Func1 func, int timeout) {
100     tryLock(key, () -> {
101         func.invoke();
102         return "Hello World!";
103     }, timeout);
104 }
105
106 /**
107  * 带返回值的
108  */
109 public <T> T tryLock(String key, Func2<T> func2, int timeout) {
110     if (StringUtils.isEmpty(key)) {
111         throw new BizException(ERROR_DISTRIBUTED_LOCK_KEY_NOT_PROVIDED);
112     }
113     Lock lock = distributedLockManager.getReentrantLock(key, DEFAULT_EXPIRY_TIME_MILLIS);
114     try {
115         if (lock.tryLock(timeout, TimeUnit.SECONDS)) {
116             try {
117                 return func2.invoke();
118             } finally {
119                 lock.unlock();
120             }
121         } else {
122             Cat.logEvent(CAT_TYPE, DUPLICATE_LOCK_KEY_MESSAGE + key);
123             throw new BizException("存在重复提交, 请稍后再试");
124         }
125     } catch (InterruptedException e) {
126         Thread.currentThread().interrupt();
127         Cat.logEvent(CAT_TYPE, "tryLock interrupted, key:" + key);
128         log.error("tryLock interrupted, key:{}", key, e);
129     }
```

```
129     }
130     return null;
131 }
132
133 public <T> T multiTryLock(String[] keyList, Func2<T> func2, int timeout) {
134     if (keyList == null || keyList.length == 0) {
135         throw new BizException(ERROR_DISTRIBUTED_LOCK_KEY_NOT_PROVIDED);
136     }
137     Lock lock = distributedLockManager.getNewMultiLock(keyList, DEFAULT_EXPIRY_TIME_MILLIS);
138     try {
139         if (lock.tryLock(timeout, TimeUnit.SECONDS)) {
140             try {
141                 return func2.invoke();
142             } finally {
143                 lock.unlock();
144             }
145         } else {
146             Cat.logEvent(CAT_TYPE, "multiTryLock duplicate key:" + JsonUtils.toJson(keyList));
147             throw new BizException("存在重复提交,请稍后再试");
148         }
149     } catch (InterruptedException e) {
150         log.error("multiTryLock-interruptedexception, keyList:{}, JsonUtils.toJson(keyList), e)",
151             Thread.currentThread().interrupt());
152         Cat.logEvent(CAT_TYPE, "multiTryLock interrupted, key:" + JsonUtils.toJson(keyList));
153     }
154     return null;
155 }
156
157 /**
158  * 带返回值的
159  * 异常时补偿
160  */
161 public <T> T tryLockWithCompensate(String key, Func2<T> logicAfterLog, Func1 logicCompensate, int timeout) {
162     if (StringUtils.isEmpty(key)) {
163         throw new BizException(ERROR_DISTRIBUTED_LOCK_KEY_NOT_PROVIDED);
164     }
165     Lock lock = distributedLockManager.getReentrantLock(key, DEFAULT_EXPIRY_TIME_MILLIS);
166     try {
167         if (lock.tryLock(timeout, TimeUnit.SECONDS)) {
168             try {
169                 return logicAfterLog.invoke();
170             } finally {
171                 lock.unlock();
172             }
173         } else {
174             Cat.logEvent(CAT_TYPE, DUPLICATE_LOCK_KEY_MESSAGE + key);
```

